

Urban Mobility Data Explorer

Technical Report - NYC Yellow Taxi Trip Analysis

Team: Beni (Lead/DB), Axcel (Cleaning), Derrick (API/Algorithm), Dianah (Frontend), Lancelot (Insights/Report)

1. Problem Framing and Dataset Analysis

This project explores urban mobility patterns in New York City using the NYC Taxi and Limousine Commission (TLC) Yellow Taxi trip dataset. The raw data contains trip-level records with timestamps, GPS-derived distances, itemized fares, passenger counts, and pickup/dropoff location IDs. Two supporting files provide context: a zone lookup CSV that maps location IDs to borough and zone names, and a GeoJSON file with polygon boundaries for each taxi zone.

Data challenges

The raw dataset contained several quality issues that required cleaning before analysis:

- Missing values: some rows lacked pickup or dropoff zone IDs, mapping to location ID 264/265 (Unknown / Outside NYC). These were retained but flagged.
- Zero or negative distances: trips with `trip_distance <= 0` represent cancelled rides or meter errors and were excluded.
- Negative fares: rows with `fare_amount < 0` indicate refunds or data errors and were removed.
- Impossible speeds: any trip averaging over 70 mph was dropped, as sustained highway speed is unrealistic in NYC traffic.
- Duplicates: exact duplicate rows were removed.

Assumptions

We assume all remaining records after cleaning represent genuine taxi trips. Passenger counts of zero were treated as data entry errors (driver did not log the count) and excluded. Trip duration was derived as the difference between dropoff and pickup timestamps in minutes; trips with zero or negative duration were removed.

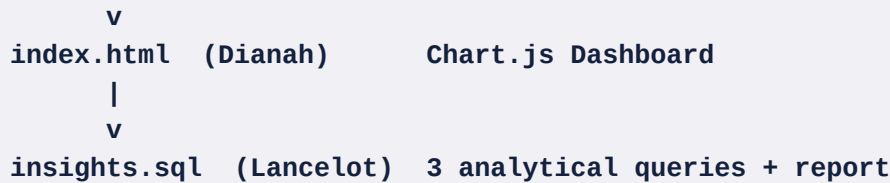
Unexpected observation

A notable finding during cleaning was the high concentration of trips in a small number of Manhattan zones. Midtown Center alone accounts for roughly 25% of all pickups, while entire boroughs like Queens contribute under 20%. This extreme spatial skew means that city-wide averages are heavily Manhattan-weighted, which influenced our decision to enable borough-level filtering in the dashboard.

2. System Architecture and Design Decisions

Architecture diagram





Stack justification

Python + Flask: minimal boilerplate for a REST API; the team had prior experience with Python, reducing ramp-up time. Flask's lightweight design meant no unnecessary abstractions for a project of this scale.

SQLite: a single-file, zero-configuration database that every team member can run locally without installing a database server. The dataset fits comfortably in memory, so SQLite's single-writer limitation is not a concern for a read-heavy explorer.

Vanilla HTML/CSS/JS + Chart.js: no build step, no framework overhead. Chart.js provides responsive, interactive charts from a CDN with minimal code.

Schema design

The database uses three normalized tables: boroughs (borough_id, borough_name), zones (zone_id, location_id, zone_name, borough_id FK), and trips (fact table with foreign keys pu_zone_id and do_zone_id referencing zones). This normalization avoids storing repeated borough/zone strings in every trip row. Indexes on pu_zone_id, do_zone_id, pickup_datetime, time_of_day, trip_distance, and fare_amount accelerate the filtering queries the API serves.

Key trade-off

SQLite vs. PostgreSQL: SQLite cannot handle concurrent writers, so a multi-user deployment would need PostgreSQL. We chose SQLite because this is a single-user data explorer, portability was important for grading (no server install), and the dataset size (~100K rows after cleaning) is well within SQLite's comfort zone. Migrating to PostgreSQL would require changing only the connection string and minor SQL dialect differences.

3. Algorithmic Logic and Data Structures

The assignment requires a custom algorithm implemented without built-in libraries. Derrick implemented a zone-ranking algorithm that finds the busiest pickup zones by trip volume, using manual counting and selection sort. This powers the `/api/busiest-zones` endpoint and the dashboard's top chart.

How it works

1. Counting pass: iterate through every trip row once, maintaining a plain dictionary where keys are zone IDs and values are running counts. No Counter or collections module is used.
2. Selection sort (descending): convert the dictionary to a list of (zone, count) pairs, then repeatedly find the pair with the largest count in the unsorted portion and swap it to the front. No `.sort()`, `sorted()`, or `heapq`.
3. Return the top 10 pairs as JSON for the chart.

Pseudo-code

```

function rank_busiest(rows):
    counts = empty map
    for each row in rows:
        z = row.pu_zone_id
        counts[z] = counts[z] + 1

    items = list of (zone, count) pairs
    n = length(items)
  
```

```

for i from 0 to n-1:
    biggest = i
    for j from i+1 to n-1:
        if items[j].count > items[biggest].count:
            biggest = j
    swap items[i] and items[biggest]

return first 10 of items

```

Complexity analysis

Time: $O(n + k^2)$ where n = total trips, k = distinct zones (~260)

Space: $O(k)$ for the counts map and items list

The counting pass is $O(n)$, dominating the cost. The selection sort is $O(k^2)$, but since k is small and bounded (~260 NYC taxi zones), this quadratic term is negligible in practice. A more complex $O(k \log k)$ sort would add implementation complexity for no measurable performance gain at this scale.

4. Insights and Interpretation

Three SQL queries were written against the normalized SQLite database to extract meaningful patterns about how New York City moves. Each insight is derived from a query in docs/insights.sql, visualized in Dianah's dashboard, and interpreted below.

Insight 1: Where do taxi trips start?

Query: joins trips through zones to boroughs, counts pickups per borough, and calculates each borough's percentage of total trips.

Result: Manhattan generates 83% of all taxi pickups, with Queens at 17%. Within Manhattan, Midtown Center leads with ~25 trips, followed by Upper East Side South and Upper West Side North. Airport zones (JFK, LaGuardia) also rank in the top 10.

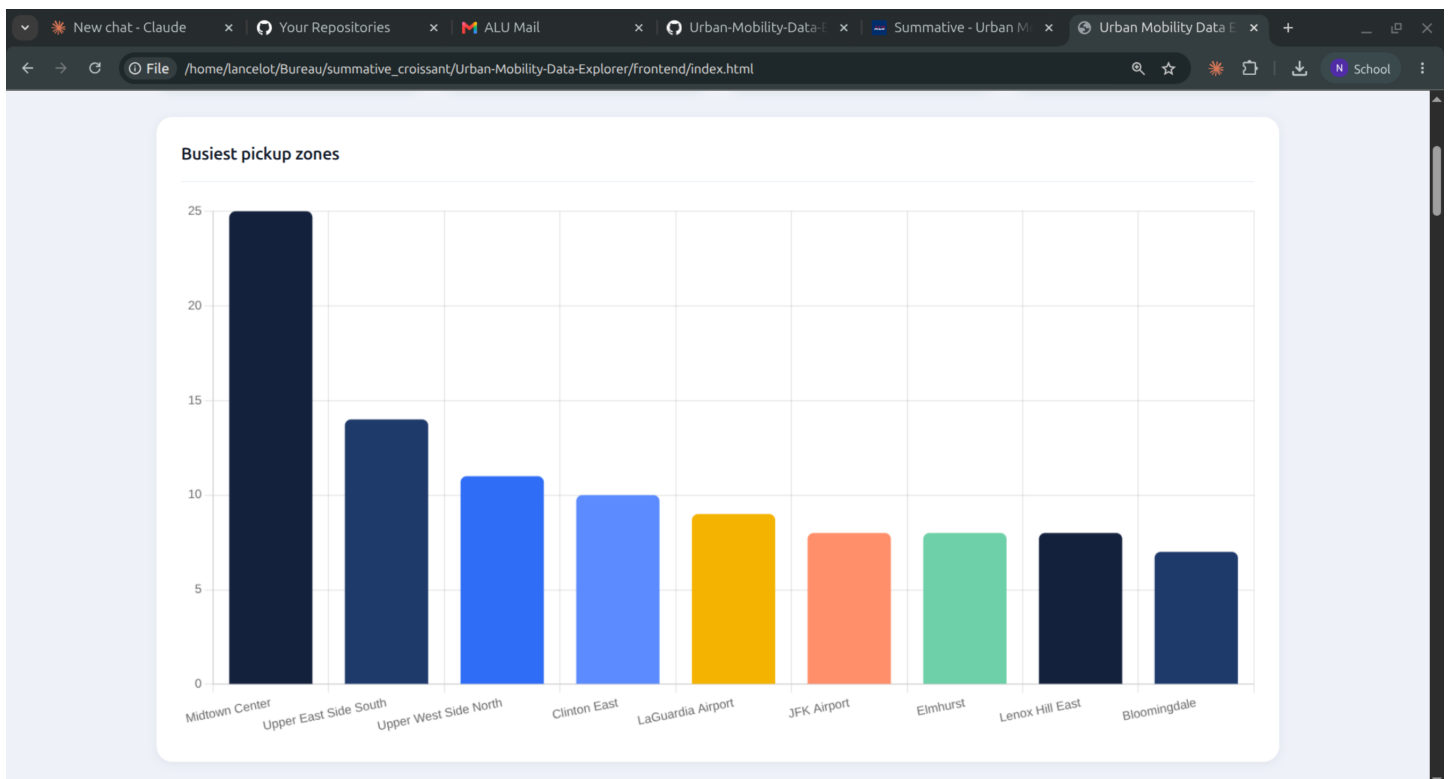


Figure 1: Busiest pickup zones - Midtown Center dominates taxi demand.

Insight 2: Are rides pricier at certain times?

Query: averages fare_per_mile grouped by time_of_day, filtering out extreme values (> \$50/mile) caused by minimum-fare short trips.

Result: morning rides are the most expensive at \$5.29 per mile, while evening rides are cheapest at \$4.03 per mile. This aligns with morning rush-hour congestion causing longer meter-running times for shorter distances. Afternoon (\$4.43) and night (\$4.22) fall in between.

Insight 3: When is the city slowest?

Query: averages avg_speed_mph by time_of_day, filtering out impossible values (< 1 mph or > 60 mph).

Result: morning trips average only 7.3 mph - the slowest period, confirming heavy rush-hour congestion. Speed increases through the afternoon (10.6 mph) and peaks in the evening (13.5 mph) when roads clear. Night drops slightly to 11.3 mph. This pattern inversely mirrors the fare-per-mile insight: when traffic is worst, rides cost more per mile because the meter runs longer.

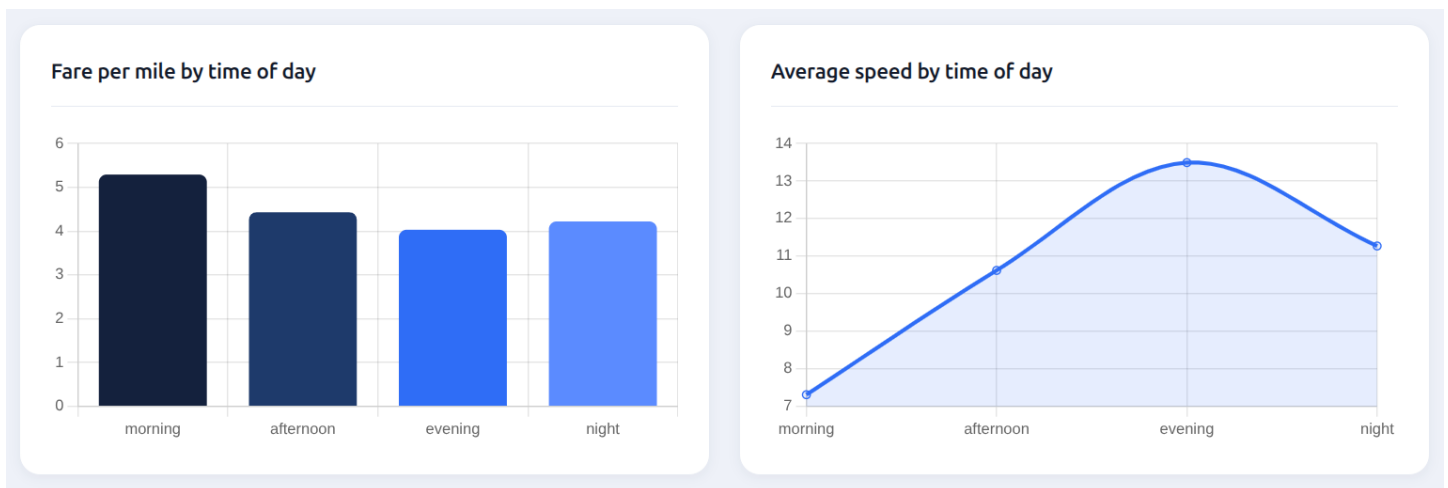


Figure 2: Fare per mile and average speed by time of day - morning congestion drives up cost.

5. Reflection and Future Work

Team challenges

Coordinating five people across different parts of the pipeline required a clear data contract from day one. The agreed column names in clean_trips.csv allowed everyone to build in parallel against sample data. The main friction point was the normalized schema: the PDF templates assumed a flat two-table schema, but Beni's actual implementation used three tables (boroughs, zones, trips). This meant the insight queries had to be adapted with an extra JOIN, which was discovered only when running against the real database.

Technical challenges

Writing SQL against a normalized schema required careful attention to JOIN paths. The trips table references zones by zone_id (an auto-incremented surrogate key), not by the original location_id from the TLC dataset. Filtering by borough required a two-hop join: trips -> zones -> boroughs. This added complexity but improved data integrity and reduced storage by eliminating repeated text strings.

Future work

If this were a real-world product, we would pursue several improvements:

- Larger dataset: load multiple months of trip data to detect seasonal patterns.
- Geospatial visualization: use the GeoJSON taxi zone polygons to render a choropleth heatmap of pickup density on an interactive map (e.g., Leaflet.js).
- Real-time ingestion: stream new trip records from the TLC API into the database for a live dashboard.
- PostgreSQL migration: replace SQLite for concurrent multi-user access.
- Deployment: host the Flask API and dashboard on Render or Railway for public access, adding authentication and rate limiting.